

LAPORAN PRAKTIKUM 3
PEMROGRAMAN BERORIENTASI OBJEK
KEAMANAN & INTEGRITAS DATA (ENKAPSULASI)



Disusun Oleh :

KINAYA NOVRYA MANDA

(2511531016)

Dosen Pengampu :

NURFIAH, S.ST, M.KOM

Asisten Praktikum :

EZZA ADDINI

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI	i
BAB 1 PENDAHULUAN	
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	2
BAB 2 PEMBAHASAN	
2.1 Tahapan Pelaksanaan Praktikum.....	3
2.1.1 Membuat Package dan Class.....	3
2.1.2 Membuat Class Rekening.....	3
2.1.3 Membuat Class Transaksi.....	7
2.1.4 Membuat Class Main.....	8
2.2 Tugas Praktikum.....	13
2.2.1 Tugas 1.....	13
2.2.2 Tugas 2.....	14
2.2.3 Output Tugas 1 dan Tugas 2.....	15
BAB 3 KESIMPULAN	
3.1 Kesimpulan.....	16
3.2 Link Repository.....	16

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pemrograman berorientasi objek, keamanan data merupakan salah satu hal yang penting agar data di dalam objek tidak dapat diakses atau diubah secara sembarangan. Salah satu konsep yang digunakan untuk menjaga keamanan data adalah enkapsulasi, yaitu dengan menyembunyikan atribut tertentu menggunakan access modifier seperti `private` dan menyediakan akses melalui method yang sesuai, seperti `getter` dan `setter`. Pada Praktikum 3 ini, konsep enkapsulasi diterapkan pada sistem perbankan dengan mengamankan data pada class `Rekening` dan `Transaksi`.

Selain penerapan enkapsulasi, praktikum ini juga menerapkan keamanan menggunakan PIN sebagai proses autentikasi. PIN digunakan saat pembuatan rekening dan harus diverifikasi sebelum user dapat melakukan transaksi tarik tunai maupun melihat riwayat transaksi. Dengan penerapan tersebut, program tidak hanya menyembunyikan data penting, tetapi juga membatasi akses terhadap fitur tertentu sehingga sesuai dengan konsep *Information Hiding* dan penggunaan access modifier dalam Java.

1.2 Tujuan Praktikum

Tujuan dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami urgensi *Information Hiding* dalam rekayasa perangkat lunak.
2. Mengimplementasikan Access Modifiers (`private`, `public`, `protected`) dengan tepat.
3. Mengontrol akses pembacaan dan penulisan atribut menggunakan `Getter` dan `Setter`.

4. Menerapkan enkapsulasi untuk memvalidasi business logic pada saat autentikasi PIN.

1.3 Manfaat Praktikum

Manfaat dilakukannya praktikum ini adalah sebagai berikut :

1. Memahami konsep *Information Hiding* dalam pemrograman berorientasi objek.
2. Memahami penggunaan access modifier private, public, dan protected.
3. Memahami penggunaan getter dan setter untuk mengatur akses terhadap data.
4. Memahami penerapan enkapsulasi dan autentikasi PIN untuk meningkatkan keamanan data pada program.

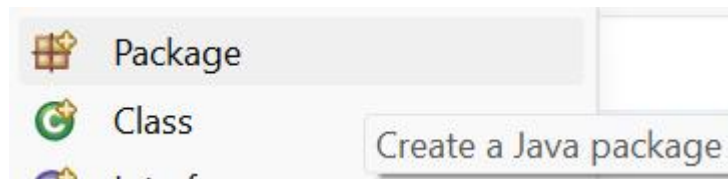
BAB II

PEMBAHASAN

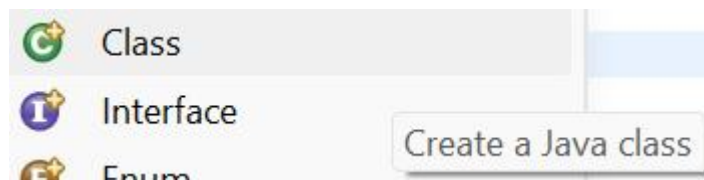
2.1 Tahapan Pelaksanaan Praktikum

2.1.1 Membuat Package dan Class

1. Buat Package dengan nama Praktikum3.



2. Setelah membuat Package, lanjutkan dengan membuat Class.



2.1.2 Membuat Class Rekening

1. Buat class baru dengan nama Rekening pada package Praktikum3. Pada class ini nantinya akan dibuat data dan method yang berhubungan dengan sebuah rekening.

Name:	Rekening
Modifiers:	☒ public ☐ package ☐ private ☐ protected ☐ abstract ☐ final ☐ static ☒ none ☐ sealed ☐ non-sealed ☐ final
Superclass:	java.lang.Object

2. Scanner dan ArrayList digunakan untuk kebutuhan input dan penyimpanan data rekening pada program.

```
package Praktikum1;  
import java.util.Scanner;  
import java.util.ArrayList;
```

3. Selanjutnya buat atribut rekening dengan menggunakan access modifier private.

```
// 1. Mengunci atribut dengan 'private'
private String nomorRekening;
private String namaPemilik;
private double saldo;
private String pin; // Data sensitif
```

Penggunaan private bertujuan untuk membatasi akses langsung dari luar class terhadap data rekening. Atribut pin juga dibuat private karena merupakan data yang digunakan untuk keamanan rekening.

4. Tambahkan atribut riwayat transaksi dengan access modifier private.

```
private ArrayList<Transaksi> riwayatTransaksi;
```

Atribut tersebut digunakan untuk menyimpan seluruh riwayat transaksi pada satu rekening. Konsep ini tetap menerapkan asosiasi 1-to-many, yaitu satu objek Rekening dapat memiliki banyak objek Transaksi.

5. Selanjutnya buat constructor dengan nama yang sama seperti nama class. Constructor ini menerima 4 parameter, yaitu nomor, nama, saldoAwal dan pinAwal.

```
// 2. Modifikasi Constructor untuk menerima PIN awal
public Rekening(String nomor, String nama, double saldoAwal,
String pinAwal) {
    this.nomorRekening = nomor;
    this.namaPemilik = nama;
    this.saldo = saldoAwal;
```

6. Setelah data rekening diinisialisasi, tambahkan proses validasi PIN pada constructor.

```
// Validasi PIN di dalam Constructor
if (pinAwal.length() == 6) {
    this.pin = pinAwal;
} else {
    System.out.println("Peringatan: PIN harus 6 digit!
Menggunakan PIN default 123456");
    this.pin = "123456";
```

Kode digunakan untuk memeriksa panjang PIN yang dimasukkan saat rekening dibuat. Jika panjang PIN adalah 6 karakter, PIN tersebut

digunakan. Jika tidak, program memberikan peringatan dan menggunakan PIN default 123456.

7. Setelah itu, inisialisasi riwayatTransaksi menggunakan ArrayList.

```
this.riwayatTransaksi = new ArrayList<>();  
    riwayatTransaksi = new ArrayList<>();  
    System.out.println("Rekening atas nama " + namaPemilik + "  
    berhasil dibuat.");
```

riwayatTransaksi diinisialisasi agar dapat digunakan untuk menyimpan transaksi yang dilakukan oleh rekening.

8. Buat getter untuk atribut yang masih diperbolehkan untuk dibaca dari luar class.

```
// 3. Getter untuk atribut yang diizinkan dibaca publik  
    public String getNomorRekening() { return nomorRekening; }  
    public String getNamaPemilik() { return namaPemilik; }
```

Getter digunakan agar data nomorRekening dan namaPemilik tetap dapat diakses oleh class lain tanpa membuka akses langsung terhadap atribut private.

9. Buat method otentikasi() untuk melakukan pemeriksaan PIN.

```
// 4. Method Otentikasi Internal (Validasi Enkapsulasi)  
    public boolean otentikasi(String inputPin) {  
        return this.pin.equals(inputPin);
```

Method otentikasi() digunakan untuk membandingkan PIN yang tersimpan pada rekening dengan PIN yang dimasukkan oleh pengguna. Jika PIN sesuai, method mengembalikan nilai true, sedangkan jika PIN tidak sesuai akan mengembalikan nilai false.

10. Buat method setorTunai()

```
    public void setorTunai(double nominal) {  
        if (nominal > 0) {  
            saldo += nominal;  
            String idTrx = "TRX-S-" + System.currentTimeMillis();  
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit",  
            nominal);  
            riwayatTransaksi.add(trxBaru);  
            System.out.println("Setor tunai Rp" + nominal + "  
            berhasil. Saldo saat ini: Rp" + saldo);  
        } else {
```

```

        System.out.println("Gagal: Nominal setor harus lebih
        dari 0!");
    }
}

```

Method `setorTunai()` digunakan untuk melakukan penyetoran uang ke rekening. Program mengecek apakah nominal setor lebih dari 0, kemudian menambahkan nominal tersebut ke saldo. Jika berhasil, transaksi dicatat sebagai Kredit dan disimpan ke dalam `riwayatTransaksi`.

11. Buat method `tarikTunai()`

```

public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println("Transaksi Gagal : Minimal
        nominal penarikan 10.000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak
        mencukupi. Saldo Anda: Rp" + saldo);
    } else { saldo -= nominal;
        System.out.println("Tarik tunai berhasil.");
        System.out.println("Jumlah penarikan: Rp" + nominal);
        System.out.println("Saldo sekarang: Rp" + saldo);

        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit",
        nominal);
        riwayatTransaksi.add(trxBaru);
    }
}

```

Method `tarikTunai()` digunakan untuk melakukan penarikan uang dari rekening. Program mengecek apakah nominal penarikan sudah memenuhi batas minimal dan saldo mencukupi. Jika penarikan berhasil, saldo akan dikurangi dan transaksi dicatat sebagai Debit ke dalam `riwayatTransaksi`.

12. Buat method `cekInformasi()`

```

public void cekInformasi() {
    System.out.println("=== INFO REKENING ===");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : Rp" + saldo);
    System.out.println("=====");
}

```

Method cekInformasi() digunakan untuk menampilkan informasi rekening yang sedang aktif. Informasi yang ditampilkan meliputi nomor rekening, nama pemilik, dan saldo akhir rekening.

13. Buat method cetakMutasi()

```
public void cetakMutasi() {  
    if (riwayatTransaksi.isEmpty()) {  
        System.out.println("Belum ada transaksi pada rekening  
        ini");  
    } else {  
        for (Transaksi transaksi : riwayatTransaksi) {  
            transaksi.cetakDetail();  
        }  
    }  
}
```

Method cetakMutasi() digunakan untuk menampilkan riwayat transaksi pada rekening. Program mengecek apakah terdapat transaksi, kemudian menggunakan perulangan for-each untuk menampilkan detail setiap transaksi melalui method cetakDetail(). Jika belum ada transaksi, program akan menampilkan pesan bahwa belum ada transaksi pada rekening tersebut.

2.1.3 Membuat Class Transaksi

1. Buat class baru dengan nama Transaksi pada package Praktikum3.

Name:	Transaksi			
Modifiers:	☒ public	☐ package	☐ private	☐ protected
	☐ abstract	☐ final	☐ static	
	☒ none	☐ sealed	☐ non-sealed	☐ final

2. Tambahkan tiga atribut yaitu idTransaksi untuk menyimpan identitas atau ID transaksi, jenis untuk menyimpan jenis transaksi seperti Debit atau Kredit, dan nominal untuk menyimpan jumlah uang yang terlibat dalam transaksi.

```
public class Transaksi {  
    String idTransaksi;
```

```
String jenis;  
double nominal;
```

3. Selanjutnya, buat constructor yang digunakan untuk menginisialisasi data transaksi saat objek Transaksi dibuat. Setiap objek transaksi memiliki ID, jenis transaksi, dan jumlah transaksi yang sesuai.

```
// Constructor  
public Transaksi(String id, String jenis, double nominal) {  
    this.idTransaksi = id;  
    this.jenis = jenis;  
    this.nominal = nominal;  
}
```

4. Setelah itu, buat method cekDetail() yang digunakan untuk menampilkan detail transaksi ke layar.

```
public void cetakDetail() {  
    System.out.println("ID: " + idTransaksi + " | Jenis: " +  
        jenis + " | Nominal: Rp" + nominal);  
}
```

2.1.4 Membuat Class Main

1. Membuat scanner dan variabel program.

```
public class Main {  
    public static void main(String[] args) {  
        Scanner input = new Scanner(System.in);  
        ArrayList<Rekening> daftarRekening = new ArrayList<>();  
        Rekening akunAktif = null; // Objek belum diinisialisasi  
        (null)  
        boolean isRunning = true;  
        String pinYangDiInput;
```

Kode di atas digunakan untuk menyiapkan input pengguna dengan Scanner, menyimpan daftar rekening menggunakan ArrayList, menentukan akun yang sedang aktif melalui akunAktif, dan mengatur program agar tetap berjalan dengan isRunning. Variabel pinYangDiInput digunakan untuk menyimpan PIN yang dimasukkan pengguna saat proses autentikasi.

2. Membuat menu utama. Perulangan while digunakan agar menu terus ditampilkan selama program masih berjalan.

```
System.out.println("=== SISTEM PERBANKAN MINI ===");

while (isRunning) {
    System.out.println("\n=== Menu Utama ===");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi Rekening");
    System.out.println("5. Ganti Akun");
    System.out.println("6. Cetak Mutasi (Riwayat)");
    System.out.println("0. Keluar");
    System.out.print("Pilih menu: ");
```

3. Input dari pengguna disimpan ke dalam variabel pilihan. Selanjutnya, switch digunakan untuk menentukan proses yang dijalankan sesuai dengan nomor menu yang dipilih.

```
int pilihan = input.nextInt();
    input.nextLine(); // Membersihkan buffer enter

    switch (pilihan) {
```

4. Membuat proses buka rekening

```
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();
    input.nextLine();

    System.out.print("Masukkan PIN (6 digit): ");
    String pin = input.nextLine();

    Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);
    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
    System.out.println("Rekening berhasil ditambahkan.");
    break;
```

Pada menu ini, pengguna memasukkan nomor rekening, nama pemilik, saldo awal, dan PIN. Setelah itu, data tersebut digunakan untuk membuat

objek Rekening. PIN diberikan ke constructor agar tersimpan pada rekening dan dapat digunakan untuk proses autentikasi.

5. Membuat proses setor tunai

```
case 2:

if (akunAktif == null) {
    System.out.println("Error: Mohon maaf, Anda belum memiliki
    nomor rekening!");
} else {
    System.out.print("Masukkan nominal setor: ");
    double setor = input.nextDouble();
    akunAktif.setorTunai(setor);
}
break;
```

Sebelum melakukan setor tunai, program mengecek apakah sudah ada rekening yang aktif. Jika belum ada, program menampilkan pesan kesalahan. Jika sudah ada, nominal setor dimasukkan dan method `setorTunai()` dipanggil.

6. Membuat proses tarik tunai dengan autentikasi PIN

```
case 3:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan PIN: ");
        pinYangDiInput = input.nextLine();

        if (akunAktif.otentikasi(pinYangDiInput)) {
            System.out.print("Masukkan nominal tarik: Rp");
            double tarik = input.nextDouble();
            akunAktif.tarikTunai(tarik);
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan
            salah!");
        }
    }
    break;
```

Pada menu tarik tunai, program terlebih dahulu mengecek apakah terdapat akun aktif. Jika ada, pengguna diminta memasukkan PIN. PIN tersebut kemudian diperiksa menggunakan method `otentikasi()`

7. Membuat proses cek informasi

case 4:

```
if (akunAktif == null) {
    System.out.println("Error: Anda belum membuka rekening!");
} else {
    akunAktif.cekInformasi();
}
break;
```

Jika ada akun aktif, program menjalankan `cekInformasi()` untuk menampilkan data rekening.

8. Membuat proses mengganti akun aktif

```
case 5 :
    if (daftarRekening.isEmpty()) {
        System.out.println("Belum ada rekening yang tersedia.");
    } else {
        System.out.print("Masukkan nomor rekening yang ingin digunakan: ");
        String nomorCari = input.nextLine();
        boolean ditemukan = false;
        for (Rekening rekening : daftarRekening) {
            if (rekening.getNomorRekening().equals(nomorCari)) {
                akunAktif = rekening;
                ditemukan = true;
                System.out.println("Berhasil mengganti akun.");
                System.out.println("Akun aktif: " + akunAktif.getNamaPemilik());
                break;
            }
        }
        if (!ditemukan) {
            System.out.println("Rekening tidak ditemukan.");
        }
    }
    break;
```

Pengguna memasukkan nomor rekening, kemudian program mencari rekening tersebut di `daftarRekening` dan menjadikannya sebagai akun aktif. Digunakan method `getNomorRekening()` dan `getNamaPemilik()` karena atribut pada class `Rekening` sudah dibuat private.

9. Membuat proses cetak mutasi dengan autentikasi PIN

```

case 6:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka rekening!");
    } else {
        System.out.print("Masukkan PIN: ");
        pinYangDiInput = input.nextLine();

        if (akunAktif.otentikasi(pinYangDiInput)) {
            akunAktif.cetakMutasi();
        } else {
            System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");
        }
    }
    break;

```

Case 6 digunakan untuk menampilkan riwayat transaksi atau mutasi rekening. Sebelum cetakMutasi() dijalankan, pengguna harus memasukkan PIN terlebih dahulu. Jika PIN benar, riwayat transaksi ditampilkan. Jika PIN salah, akses ditolak dan riwayat transaksi tidak ditampilkan. Ketentuan ini sesuai dengan tugas modul yang meminta autentikasi sebelum menu 6 dijalankan.

10. Membuat proses menghentikan program

```

case 0:
    isRunning = false;
    System.out.println("Sistem ditutup. Terima kasih!");
    break;

default:
    System.out.println("Pilihan tidak valid!");

```

Default digunakan untuk menampilkan pesan kesalahan jika pengguna memasukkan pilihan menu yang tidak tersedia.

11. Program diakhiri dengan menutup scanner

```

input.close();

```

2.2 Tugas Praktikum

2.2.1 Tugas 1 (Menu 1 : Buka Rekening)

Lakukan perubahan pada Menu 1 atau proses buka rekening dengan menambahkan input PIN sebelum objek Rekening dibuat. PIN yang dimasukkan pengguna kemudian dikirimkan ke constructor Rekening sebagai PIN awal. Hal ini dilakukan agar setiap rekening yang dibuat memiliki PIN yang dapat digunakan untuk proses autentikasi.

➤ Langkah Kerja :

1. Buka class Main dan cari bagian case 1 yang digunakan untuk proses membuka rekening baru.
2. Tambahkan input PIN setelah user memasukkan saldo awal. PIN yang dimasukkan disimpan ke dalam variabel pin.

```
System.out.print("Masukkan PIN (6 digit): ");  
String pin = input.nextLine();
```

Kode tersebut digunakan untuk meminta user memasukkan PIN yang akan digunakan sebagai keamanan rekening. PIN disimpan dalam bentuk String agar dapat digunakan sebagai input PIN 6 digit.

3. Tambahkan PIN sebagai parameter constructor.

```
Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);
```

PIN yang telah dimasukkan user diteruskan ke constructor Rekening saat objek rekening dibuat. Dengan begitu, setiap rekening memiliki PIN masing-masing.

2.2.2 Tugas 2 (Menu 3 dan 6 : Keamanan Transaksi)

Perbarui program agar pengguna harus memasukkan PIN saat membuat rekening, melakukan Tarik Tunai, dan Cetak Mutasi. PIN yang dimasukkan akan diperiksa menggunakan method `otentikasi()`. Jika PIN benar, proses dapat dilanjutkan, sedangkan jika PIN salah, akses akan ditolak dan transaksi dibatalkan.

➤ Langkah Kerja :

1. Buka bagian case 3 pada `Main.java` yang digunakan untuk proses tarik tunai.
2. Tambahkan permintaan PIN sebelum nominal penarikan dimasukkan.

```
System.out.print("Masukkan PIN: ");  
pinYangDiInput = input.nextLine();
```

Kode tersebut digunakan untuk meminta PIN dari user sebelum proses tarik tunai dilakukan.

3. Melakukan autentikasi PIN menggunakan method `otentikasi()`.

```
if (akunAktif.otentikasi(pinYangDiInput)) {  
    System.out.print("Masukkan nominal tarik: Rp");  
    double tarik = input.nextDouble();  
    akunAktif.tarikTunai(tarik);  
} else {  
    System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");  
}
```

J

Jika PIN yang dimasukkan benar, program akan melanjutkan proses tarik tunai. Jika PIN salah, transaksi dibatalkan dan program menampilkan pesan “Akses Ditolak: PIN yang Anda masukkan salah!”.

4. Mengamankan Menu 6 dengan cara yang sama, yaitu meminta PIN sebelum menampilkan riwayat transaksi.

```
System.out.print("Masukkan PIN: ");  
pinYangDiInput = input.nextLine();  
  
if (akunAktif.otentikasi(pinYangDiInput)) {  
    akunAktif.cetakMutasi();  
} else {  
    System.out.println("Akses Ditolak: PIN yang Anda masukkan salah!");  
}
```

Pada menu 6, user harus memasukkan PIN terlebih dahulu. Jika PIN benar, method cetakMutasi() dijalankan untuk menampilkan riwayat transaksi. Jika PIN salah, akses ke riwayat transaksi ditolak.

2.2.3 Output Tugas 1 dan Tugas 2

```
=== Menu Utama ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 3
Masukkan PIN: novrya
Masukkan nominal tarik: Rp300000
Tarik tunai berhasil.
Jumlah penarikan: Rp300000.0
Saldo sekarang: Rp200000.0

=== Menu Utama ===
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Cetak Mutasi (Riwayat)
0. Keluar
Pilih menu: 6
Masukkan PIN: novrya
ID: TRX-T-1790177249060 | Jenis: Debit | Nominal: Rp300000.0
```

Output tersebut menunjukkan bahwa user berhasil melakukan tarik tunai sebesar Rp300.000 setelah memasukkan PIN yang benar, sehingga saldo rekening berkurang menjadi Rp200.000. Selanjutnya, saat memilih Menu 6, user kembali memasukkan PIN yang benar dan sistem berhasil menampilkan riwayat transaksi dengan jenis Debit dan nominal Rp300.000, sehingga transaksi penarikan tersebut tercatat dalam mutasi rekening.

BAB III

KESIMPULAN

3.1 Kesimpulan

Pada Praktikum 3 ini, telah diterapkan konsep enkapsulasi dan Information Hiding pada sistem perbankan yang dibuat. Atribut penting pada class Rekening dan Transaksi diamankan dengan access modifier private, kemudian akses terhadap data dilakukan melalui method yang telah disediakan. Selain itu, dibuat juga method otentikasi() untuk melakukan pengecekan PIN sebagai bagian dari keamanan data. Hal ini sesuai dengan tujuan praktikum untuk memahami penggunaan access modifier, getter/setter, dan penerapan enkapsulasi.

Penerapan keamanan juga dilakukan pada Main.java dengan menambahkan autentikasi PIN pada Menu 1, Menu 3, dan Menu 6. Pada Menu 1, user harus memasukkan PIN ketika membuat rekening, sedangkan pada Menu 3 dan Menu 6 PIN harus diverifikasi terlebih dahulu sebelum tarik tunai atau melihat riwayat transaksi dapat dilakukan. Jika PIN yang dimasukkan salah, sistem akan menolak akses sesuai aturan pada modul. Dengan demikian, praktikum ini memberikan pemahaman bahwa enkapsulasi tidak hanya digunakan untuk menyembunyikan data, tetapi juga dapat diterapkan untuk membantu menjaga keamanan akses pada suatu program.

3.2 Link Repository

https://github.com/kinayanovryamanda7-blip/PrakPBO_B_2511531016